

RAG

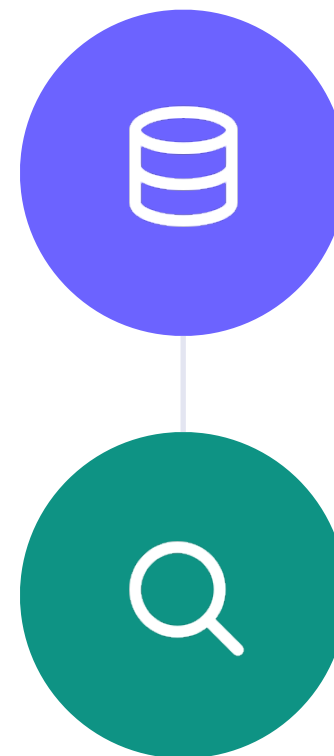
Retrieval-Augmented Generation

검색으로 찾고, 생성으로 답한다 — 내 문서로 똑똑해지는 LLM

LangChain

HuggingFace

FAISS



한 문장으로 RAG



RAG = 검색(Retrieval) + 생성(Generation)

질문과 관련된 문서를 먼저 찾아서, LLM에게 질문과 함께 건네준다. 그러면 학습하지 않은 내용도 근거 있게 답한다.

오늘 우리가 만들 것 — 3단계

1



RAG 이론

왜 필요한지 · 어떻게 동작하는지
파이프라인 전체 흐름 이해

2



FAISS

벡터를 저장하고 빠르게 검색하는
벡터 데이터베이스

3



LangChain + 코드

HuggingFace · FAISS를 엮어
실제 RAG를 코드로 구현



PART 1

RAG 이론

왜 필요하고, 어떤 단계로 동작하는가

RAG · 검색 증강 생성



LLM이 답을 만들기 전에, 외부 지식(문서)에서 관련 내용을 검색해 함께 참고하게 하는 방법

쉽게 말하면 — 오픈북 시험



기존 LLM

closed-book

외운 것(학습 데이터)만으로 답한다.
모르면 그럴듯하게 지어낸다.



RAG

open-book

관련 페이지를 펴서 보고 답한다.
근거가 있어 정확하고 출처도 남는다.

LLM의 3가지 한계

RAG는 이 세 가지 문제를 해결하기 위해 등장했다.



환각 (Hallucination)

모르는 것도 자신 있게 지어낸다. 사실이 아닌 답을 진짜처럼 말한다.



최신 정보 부족

학습 시점 이후의 사건·자료는 알지 못한다. 어제 발표된 내용은 모른다.



내 문서를 모름

회사 내부 문서·개인 자료 등 학습에 없던 데이터는 답할 수 없다.



핵심 LLM은 '아는 것'에 강하지만 '모르는 것'에 약하다. 모르는 것을 **찾아서 보여주면** 약점이 사라진다 → 그게 RAG.

RAG 파이프라인

문서를 미리 정리해 저장(인덱싱)해 두고, 질문이 오면 검색해서 LLM에게 함께 준다.



읽는 법 ①~④는 문서를 한 번만 정리해 **벡터 DB(FAISS)**에 저장하는 준비 단계. ⑤~⑥은 사용자가 질문할 때마다 일어나는 단계로, 검색된 문서 조각을 근거로 **LLM이** 답한다. 오늘 코드도 정확히 이 순서대로 짤다.

임베딩 · Embedding

한 줄 정의 텍스트(단어·문장)를 의미를 담은 숫자 벡터로 바꾸는 것. 의미가 비슷하면 벡터도 가까워진다.

텍스트 → 벡터

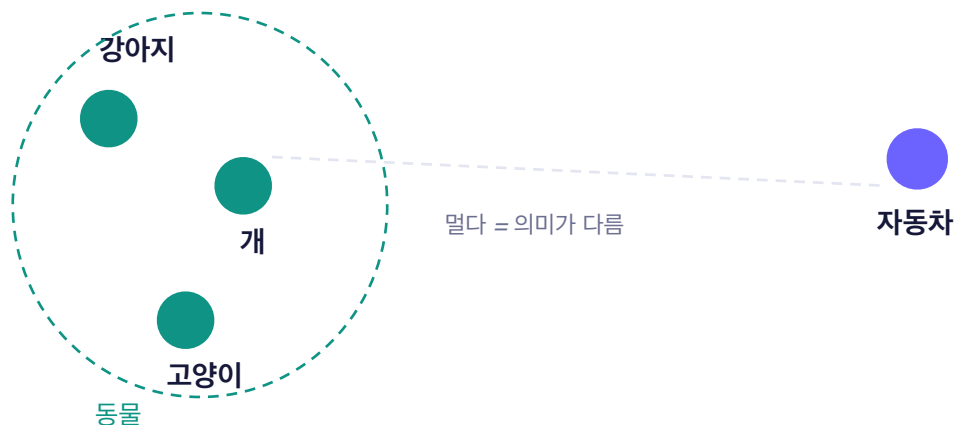
"강아지" → [0.91, 0.12, 0.78, ...]

"개" → [0.89, 0.15, 0.74, ...]

"자동차" → [0.04, 0.88, 0.21, ...]

보통 수백~수천 개의 숫자(차원)로 표현

의미 공간 — 가까울수록 비슷한 뜻



유사도 검색 · Similarity Search

결론 먼저 질문도 똑같이 벡터로 바꾼 뒤, 저장된 문서 벡터들과 **가장 가까운 top-k** 조각을 골라온다.

1



질문을 벡터로

사용자 질문을 같은 임베딩 모델로 변환

2



거리(유사도) 계산

코사인 유사도로 모든 문서 벡터와 비교

3



top-k 선택

가장 가까운 3~5개 조각만 추려서 전달

코사인 유사도 (Cosine Similarity)

두 벡터가 이루는 각도로 비슷한 정도를 잴다. 1에 가까울수록 비슷, 0에 가까울수록 무관.

$$\text{sim}(\text{질문}, \text{문서}) = \cos\theta$$

검색 결과로 답 만들기

결론 먼저 검색해 온 문서 조각을 질문과 함께 프롬프트에 넣어 LLM에게 준다 → 근거 기반 답변 완성.



근거 있는 답변 (Grounded Answer)

- 검색된 실제 문서를 근거로 답해 **환각이 줄어든다**
- 출처를 함께 제시할 수 있다
- 문서만 바꾸면 최신 정보도 즉시 반영

RAG와 파인튜닝

둘 다 LLM에 지식을 더하지만, 방법과 쓰임이 다르다. RAG는 '찾아 보여주기', 파인튜닝은 '다시 가르치기'.

항목	RAG	파인튜닝
지식 추가 방식	검색해서 그때그때 참고	모델을 다시 학습시켜 내재화
새 정보 반영	문서만 교체하면 즉시 반영	다시 학습해야 함 (느림·비용↑)
출처 제시	가능 (어느 문서인지 추적)	어려움
비용·난이도	비교적 쉽고 저렴	데이터·GPU·시간 필요
적합한 경우	자주 바뀌는 지식·내 문서 QA	말투·형식·도메인 표현 고정



PART 2

FAISS

벡터를 저장하고 빠르게 찾아주는 엔진

FAISS



Facebook AI Similarity Search

수많은 벡터 중에서 '가장 비슷한 벡터'를 매우 빠르게 찾아주는 오픈소스 라이브러리

RAG 안에서 FAISS의 역할



저장 (Indexing)

문서 조각의 임베딩 벡터를 모아 인덱스로 보관



검색 (Search)

질문 벡터와 가까운 top-k 벡터를 즉시 반환

왜 FAISS가 필요한가



문제 문서가 수만~수백만 조각이면, 질문 벡터를 **전부 하나씩 비교**하기엔 너무 느리다. 검색 한 번에 몇 초씩 걸리면 서비스가 안 된다.

FAISS의 해결책



빠른 근사 검색

전수 비교 대신 인덱스 구조로 후보를 좁혀
수백만 개도 밀리초 단위 검색



대용량 처리

수십억 벡터까지 확장 가능. 메모리·디스크
효율적으로 관리



검증된 표준

현업에서 가장 널리 쓰이는 벡터 검색 라이브러리 중 하나

FAISS 인덱스

핵심 FAISS는 벡터들을 '인덱스(index)'라는 자료구조에 담는다. 인덱스 종류에 따라 **속도와 정확도**가 달라진다.



IndexFlatL2

가장 단순 — 모든 벡터와 정확히 거리 비교

정확도 100% · 데이터 적을 때 적합 (오늘 실습용)



IVF · HNSW

후보를 미리 묶어두고 근처만 탐색

조금 부정확하지만 매우 빠름 · 대용량용

거리(distance)로 '가까움'을 잴다

L2 (유클리드 거리) : 두 벡터 사이 직선 거리. 작을수록 비슷. **Cosine (코사인)** : 방향(각도) 유사도. 클수록 비슷.

→ 오늘 실습은 가장 직관적인 *IndexFlatL2*를 *LangChain*이 자동으로 만들어 준다. 직접 만들 필요는 없다.



PART 3

LangChain & 코드

블록을 조립해 RAG를 직접 구현한다

LangChain



LLM 앱을 만드는 개발 프레임워크

로더·분할·임베딩·벡터DB·LLM 같은 부품을 레고처럼 연결해 RAG를 빠르게 조립한다



복잡한 연결을 단순화

문서→벡터→검색→LLM을 몇 줄로 이어준다



부품 교체가 자유

임베딩·벡터DB·LLM을 코드 거의 그대로 갈아 끼움



다양한 도구 지원

FAISS·HuggingFace 등 수많은 도구를 표준 방식으로 연동

RAG를 이루는 6개 블록

LangChain은 RAG를 이 6개 부품의 조립으로 본다. 오늘 코드도 위→아래 순서로 이 블록들을 만든다.



Document Loader

1

PDF·텍스트 등 원본 문서를 불러온다



Text Splitter

2

문서를 적당한 크기 조각(chunk)으로 자른다



Embeddings

3

조각을 숫자 벡터로 변환 (HuggingFace)



Vector Store

4

벡터를 저장하고 검색 (FAISS)



Retriever

5

질문에 맞는 조각을 꺼내오는 검색기



Chain (LLM)

6

검색 결과+질문을 LLM에 보내 답 생성

오늘의 기술 스택

세 가지 도구를 LangChain으로 엮는다. 전부 무료·오픈소스라 API 키 없이 로컬에서 돌릴 수 있다.



LangChain

전체 조립·연결 (프레임워크)



HuggingFace

임베딩 모델 + LLM 제공



FAISS

벡터 저장 + 유사도 검색

사용할 모델 (HuggingFace, 키 불필요)

임베딩 all-MiniLM-L6-v2
가볍고 빠른 문장 임베딩 모델

LLM google/flan-t5-base
작고 공개된 생성 모델 (데모용)

설치와 импорт

필요한 라이브러리를 설치

bash

터미널에서 한 번만 설치

```
pip install langchain langchain-community langchain-huggingface  
pip install faiss-cpu sentence-transformers pypdf transformers
```

python · imports

1) 필요한 도구 불러오기

```
from langchain_community.document_loaders import PyPDFLoader  
from langchain_text_splitters import RecursiveCharacterTextSplitter  
from langchain_huggingface import HuggingFaceEmbeddings, HuggingFacePipeline  
from langchain_community.vectorstores import FAISS  
from langchain.chains import RetrievalQA  
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline
```

문서 로드와 청크 분할

python

```
# 2) PDF 문서 불러오기
loader = PyPDFLoader("mydoc.pdf")
documents = loader.load()

# 3) 긴 문서를 조각(chunk)으로 자르기
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,      # 한 조각 글자 수
    chunk_overlap=50,    # 조각 간 겹침
)
chunks = splitter.split_documents(documents)
print(len(chunks), "개의 조각 생성")
```

무슨 일이 일어나나



PyPDFLoader PDF를 읽어 텍스트로 변환



chunk_size 조각 하나의 최대 글자 수. 너무 크면 검색이 뭉툭, 너무 작으면 의미가 잘림



overlap 조각 경계에서 문맥이 끊기지 않게 살짝 겹쳐서 자름

비유 두꺼운 책을 검색하기 좋게 '포스트잇 단위'로 잘라두는 단계. 이후 각 조각을 따로 벡터로 만든다.

임베딩과 FAISS 저장

python

```
# 4) 임베딩 모델 준비 (HuggingFace)
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# 5) 조각들을 벡터로 만들어 FAISS에 저장
vectorstore = FAISS.from_documents(chunks, embeddings)

# 저장/재사용도 가능
vectorstore.save_local("faiss_index")
```

이 한 줄의 의미

FAISS.from_documents()

가 한 번에 두 가지를 처리한다:

- ① 모든 조각을 임베딩으로 변환
- ② 그 벡터들로 FAISS 인덱스 생성

→ 이제 검색할 준비 완료!



조각(chunk)



임베딩 모델



벡터



FAISS 인덱스

검색기와 LLM 준비

python

```
# 6) 검색기(retriever) 만들기
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3}    # 가까운 조각 3개
)

# 7) HuggingFace LLM 준비 (flan-t5-base)
name = "google/flan-t5-base"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForSeq2SeqLM.from_pretrained(name)
gen = pipeline("text2text-generation",
    model=model, tokenizer=tokenizer, max_new_tokens=256)
llm = HuggingFacePipeline(pipeline=gen)
```

두 부품을 준비



Retriever

FAISS를 감싸 '질문→조각 3개'를 꺼내주는 검색기.
k=3은 몇 개 가져올지.



LLM

검색된 조각을 읽고 답을 쓰는 생성 모델.
HuggingFace pipeline을 LangChain용으로 감쌌다.

RAG 체인과 질의응답

python

```
# 8) 검색 + 생성을 하나로 묶기
qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True # 출처도 같이
)

# 9) 질문하기
result = qa.invoke({"query": "이 문서의 핵심은?"})
print(result["result"]) # 답변
print(result["source_documents"]) # 근거 문서
```



RetrievalQA가 하는 일

- ① retriever로 질문과 가까운 조각 검색
- ② 조각 + 질문을 프롬프트로 합침
- ③ LLM이 그 근거로 답 생성
- ④ 답변 + 출처 문서 반환

완성 이 9단계가 전부다. 로드 → 분할 → 임베딩 → FAISS → 검색 → 생성 순서가 코드에 그대로 담겨 있다.

코드와 파이프라인 한눈에

앞에서 배운 이론의 각 단계가 어떤 코드 한 줄에 해당하는지 매핑.

1		문서 로드	→	<code>PyPDFLoader(...).load()</code>
2		청크 분할	→	<code>splitter.split_documents(...)</code>
3		임베딩	→	<code>HuggingFaceEmbeddings(...)</code>
4		벡터 저장	→	<code>FAISS.from_documents(...)</code>
5		유사 검색	→	<code>vectorstore.as_retriever(...)</code>
6		답변 생성	→	<code>RetrievalQA(...).invoke(...)</code>

오늘의 핵심 3가지



1. RAG = 검색 + 생성

모르는 것을 찾아서 LLM에게 주면, 근거 있고 최신인 답이 나온다.



2. FAISS = 벡터 검색 엔진

문서 조각의 벡터를 저장하고, 질문과 가까운 조각을 빠르게 찾아준다.



3. LangChain = 조립 도구

로드·분할·임베딩·검색·생성 블록을 몇 줄로 이어 RAG를 완성한다.



다음 차시 7차시 · AI Agent & Tool Calling — RAG를 넘어 스스로 도구를 골라 쓰는 에이전트로 확장한다.